

概览

本记录用于实验提交中的「AI 协作记录 (Lab3-2)」 。 内容包含：规范化协作流程、V0-V6 版本迭代的关键决策与落地结果、主要文件与配置要点、以及典型问题排查。

全局约束

- **推荐栈**：前端 Vue 3 + Vite；后端 FastAPI (Python) ； SQLite 持久化。
- **前后端边界**：前端负责交互与展示；后端负责数据持久化、第三方服务代理 (天气/LLM) 、安全控制。
- **密钥安全**：
 - 前端只放地图 JS Key (`.env.local` 本地文件，不提交) 。
 - 天气 Web 服务 Key 与 LLM Key 仅放后端 `.env` (本地文件，不提交) 。
- **迭代方式**：每轮只做一个可验收闭环 (输入→处理→输出) ，避免一次性堆功能。

Rules (AI 职责与规范)

- 文件：`rules/lab3-2-ai职责与规范.mdc`
- 核心点：
 - 不把 LLM/天气 Key 放进前端代码或提交到仓库
 - 每个版本只推进一个目标并写清验收点
 - 先跑通再优化 (功能正确优先于 UI 微调)

V0：系统设计 (模块 / 数据 / API)

- 输出文件：`v0_lab3-2_系统设计.md`
- 关键决策：
 - SQLite 三张表：`plans`、`places`、`itinerary_items`
 - REST API：按 `plan_id` 组织资源 (规划→地点→行程→天气→AI 总结)
 - 天气与 LLM 统一由后端代理，前端只请求 `/api/...`

V1：项目初始化 (Vue 3 + FastAPI 工程结构)

- 前端：`frontend/` (Vite dev server, 代理 `/api` 到后端)
- 后端：`backend/` (FastAPI + SQLite 初始化)

V2：规划管理 (Plan CRUD + SQLite 持久化)

- 目标：规划可创建/列表/详情/更新，重启后仍存在
- 后端主要文件：
 - `backend/db.py`：建表、连接
 - `backend/schemas.py`：Pydantic 模型
 - `backend/main.py`：`/api/plans` CRUD

V3：前端规划表单与列表 (联调后端)

- 目标：在一个页面完成“规划列表 → 打开/编辑 → 保存”
- 前端主要文件：
 - `frontend/src/pages/PlanPage.vue`

- `frontend/src/api/plans.js`

V4: 地点管理 (地图选点 → 加入规划 → 列表展示)

- 目标: 高德地图可点选, 逆地理编码拿到地址与 `adcode`, 可加入规划并在列表展示
- 前端:
 - `frontend/.env.local`: `VITE_AMAP_KEY`、`VITE_AMAP_SECURITY_JS_CODE` (不提交)
 - `frontend/src/pages/PlanPage.vue`: 地图加载、点击选点、加入地点
 - `frontend/src/api/places.js`
- 后端:
 - `backend/main.py`: `/api/plans/{plan_id}/places` (list/add/delete)

V5: 天气展示 (后端统一调用高德天气)

- 目标: 地点列表可显示实时天气; 选点时可立即显示天气 (若有 `adcode`)
- 后端:
 - `backend/weather.py`: 高德天气调用 + 简单缓存 + 错误处理
 - `backend/main.py`:
 - `GET /api/weather/live?adcode=...`
 - `GET /api/plans/{plan_id}/weather/live` (按 place 聚合)
- 前端:
 - `frontend/src/api/weather.js`
 - `frontend/src/pages/PlanPage.vue`: 地点列表天气 chip、选点天气展示

V6: 行程安排 + AI 总结 (基于规划上下文)

- 目标:
 - 行程按上午/下午/晚上分配并可保存
 - AI 总结能读取完整规划上下文生成建议
- 后端:
 - `backend/main.py`:
 - `GET/PUT /api/plans/{plan_id}/itinerary`
 - `POST /api/plans/{plan_id}/ai/summary`
 - `backend/llm.py`: OpenAI 兼容 `chat/completions`, 增强错误信息与响应解析兼容性
- 前端:
 - `frontend/src/api/itinerary.js`
 - `frontend/src/api/ai.js`
 - `frontend/src/pages/PlanPage.vue`: 行程 UI + AI 总结 UI

扩展: 规划导出 + 规则检查

扩展 1: 规划导出 (Markdown / JSON)

- 后端: `GET /api/plans/{plan_id}/export?format=md|json`
 - `backend/exporter.py`: 导出模板 (MD/JSON)
 - `backend/main.py`: 接口与数据组装
- 前端: 顶部栏按钮「导出 MD」「导出 JSON」
 - `frontend/src/api/export.js`

- `frontend/src/pages/PlanPage.vue`

扩展 2：规则检查（预算 / 安排完整性 / 雨天风险）

- 后端：GET `/api/plans/{plan_id}/checks`
 - `backend/checks.py`: 结构化 issues (`level/code/title/detail`)
 - `backend/main.py`: 接口与数据组装 (包含天气风险判断)
- 前端：「AI 总结」区域增加「规则检查」按钮 + 结果面板
 - `frontend/src/api/checks.js`
 - `frontend/src/pages/PlanPage.vue`

关键排错记录

- 地图不显示：
 - 前端 `.env.local` 未配置或 dev server 未重启 → 检查 `VITE_AMAP_KEY`、`VITE_AMAP_SECURITY_JS_CODE`，重启 `npm run dev`
 - 高德 Referer 白名单未包含本地域名 (如 `localhost`) → 到控制台配置
- Key 不应提交：
 - `.env`、`.env.local` 写入 `.gitignore`；仓库只提供 `.env.example` / `.env.local.example`
- LLM 400/404/空内容：
 - 模型名不对、base_url 不对、Authorization 缺失
 - 通过增强 `backend/llm.py` 的 upstream 错误信息与响应解析，定位配置问题更快

skill 使用建议

- **explore**: 用于“定位文件/入口/原因”的只读排查 (地图不显示、找调用链、看工程结构)
- **generalPurpose**: 用于“落地一个版本闭环”的实现 (V2/V4/V5/V6、导出/规则检查)
- **browser-use**: 用于“UI/交互验收” (按脚本走: 新建→选点→加入→排时间→导出→检查→AI 总结)
- **cursor-guide**: 用于“Cursor/Rules/设置类问题” (rules、.cursor、IDE 行为)

不同环节的 Skills使用

V0 (系统设计)

- **skill**: `explore` (摸清现有文件位置) + `generalPurpose` (输出结构化设计稿)
- **适用问题**:
 - “怎么拆模块，前后端边界怎么定？”
 - “表结构与 API 怎么设计才方便迭代？”

V1 (工程初始化)

- **skill**: `generalPurpose`
- **适用问题**:
 - “前端如何代理到后端？”
 - “后端如何 `init_db` + `health` 检查？”

V2 (规划 CRUD)

- **skill**: `generalPurpose` (实现接口 + SQLite 持久化) + `explore` (快速定位 schema/SQL)

- **适用问题:**
 - “哪些字段该可选? 更新用 PUT 怎么做?”
 - “如何保证重启后数据还在?”

V3/V4 (页面主流程 + 地图选点)

- **skill:** generalPurpose (实现交互闭环) + browser-use (验收 UI/交互是否顺畅)
- **适用问题:**
 - “地图不显示/点击没反应/逆地理编码拿不到 adcode”
 - “空状态、加载中、失败提示要怎么写才不啰嗦?”

V5 (天气整合)

- **skill:** generalPurpose (后端代理第三方 + 前端展示) + explore (查调用链与错误来源)
- **适用问题:**
 - “天气应该放后端还是前端? Key 怎么管理?”
 - “地点列表按 place_id 聚合天气怎么设计返回结构?”

V6 (行程 + AI 总结)

- **skill:** generalPurpose (端到端打通) + browser-use (按脚本验收全流程)
- **适用问题:**
 - “行程分配的最小数据结构是什么?”
 - “AI 总结如何注入上下文, 如何处理 LLM 返回结构差异?”

扩展

- **skill:** generalPurpose (快速做出可展示的扩展) + explore (复用已有数据/接口)
- **适用问题:**
 - “导出 md/json 的内容结构怎么定才像‘提交件’?”
 - “规则检查怎么做到结构化、可解释, 而不是随口提醒?”

debug

- **skill:** explore (定位配置/入口) + generalPurpose (给出可操作排查序列)
- **适用问题:**
 - “为什么 Authorization 缺失 / model 不存在 / base_url 填错?”
 - “env 改了但不生效为什么? 要不要重启服务?”

真实对话摘录

- **用户:** 阅读 lab3_02_knowledge_v1.pdf 和 lab3_02_v2.pdf, 告诉我规范化 ai 操作应该有哪些步骤, 给我写出一步步版本的需求文档和详细一点的提示 (我该做什么)。
- **AI:** 整理推荐步骤, 并按 V0-V6 输出“你该做什么 / 给 AI 的提示词 / 验收标准”, 把复杂任务拆成可验收闭环。
- **用户:** 现在无法显示地图, 为什么?

- **AI**: 优先检查 `frontend/.env.local` 的 `VITE_AMAP_KEY` 与 `VITE_AMAP_SECURITY_JS_CODE`, 并重启 dev server; 再检查高德控制台 Referer 白名单是否包含 `localhost`。
- **用户**: 生成失败: 模型不存在, 请检查模型代码。
- **AI**: 这是 `LLM_MODEL` 配置错误, 换成有效模型名 (例如 `glm-4-flash`), 并确认 `LLM_BASE_URL` 指向兼容 `chat/completions` 的地址。